

Diffusion Models

Nagesh

May 2026

”In the world of noise , if you know to denoise your mind step by step , day by day , at some point you will be left with clarity.

Untouched , Uncontaminated, Natural living emerges there :) ”
(Nagesh)

Contents

Introduction	8
From VAE to Hierarchical VAE	8
Hierarchical VAE	8
DDPM as a Hierarchical VAE	9
1. Multiple Latent Spaces	9
2. Latent Dimension Equals Data Dimension	9
3. The Encoder (Forward Process) is Fixed	9
4. Only the Decoder is Learned	10
Summary	11
DDPM Formulation	11
Notation	11
Forward Process	12
Forward Process as a Recurrent Equation	13
Derivation of Closed Form	13
Forward Process as a Markov Chain	14
Reverse Process	15
Interpretation	15
ELBO Formulation	16
ELBO for DDPM	16
Factorization of the Reverse Process	17
Factorization of the Forward Process	17

Substituting into the ELBO	17
Why Rewrite the Forward Process?	18
Applying Bayes Rule	18
Substituting into the ELBO	19
Final ELBO	20
Rewriting the ELBO Using Conditional Expectations	20
Conditional Expectation Property	21
Writing the ELBO in Terms of KL Divergence	21
Interpretation of the ELBO Terms	21
1. Reconstruction Term	22
2. Prior Matching Term	22
3. Denoising Consistency Term	22
Summary of ELBO Terms	23
Optimization Objective	23
Computing the ELBO Terms	23
Ignoring the Prior Matching Term	24
Computing the Reverse Posterior	24
Known Forward Distributions	24
Deriving the Closed Form of $q(x_t x_0)q(\mathbf{x}_{t-1} \mathbf{x}_0)$	25
Deriving $q(x_{t-1} x_t, x_0)q(\mathbf{x}_{t-1} \mathbf{x}_t, \mathbf{x}_0)$	26
KL Divergence in the Denoising Term	26
Computing the Reconstruction Term	27
Final DDPM Objective	28
Neural Network Parameterization	28
DDPM as a Regressor over the True Data	29
Parameterizing the Reverse Mean	29
Substituting into the KL Objective	30

Interpretation	30
Important Conceptual Insight	31
DDPM as a Denoising Regressor	31
Training DDPM	32
Training Procedure	32
Mini-batch Training	33
Forward and Backward Pass	33
Forward Pass	33
Backward Pass	33
Important Interpretation	34
Why DDPM Training is Stable	34
Inference (Sampling) in DDPM	34
Reverse Sampling Process	35
Learned Reverse Distribution	35
Reverse Mean Parameterization	35
Sampling Algorithm	36
Important Computational Observation	37
Why DDPM Sampling is Slow	37
Important Note on Hyperparameters	37
Other interpretations of DDPMs	38
DDPM as a Noise Predictor	38
Different Parameterizations of DDPM	38
Conditional Generation	39
Noise Prediction Formulation	39
Forward Process	39
Expressing x_0 in Terms of Noise	40

Consistency Term	40
Substituting x_0	41
Parameterizing the Reverse Mean Using Noise Prediction	41
Consistency Term in Noise Space	42
Final Noise Prediction Formulation	42
Interpretation	42
DDPM as a Score Predictor	43
Tweedie's Formula	43
Interpretation of the Score Function	43
Derivation of Tweedie's Formula	44
Applying Tweedie's Formula to DDPM	45
Expressing the Reverse Mean Using the Score Function	46
Learned Score Function	47
Consistency Term in Score Space	47
True Score and Noise Relationship	47
Important Insight	48
Important Observation	48
Four Interpretations of DDPM	49
Final Remark	49
Guided Diffusion Models	49
Objective of Conditional Diffusion Models	50
Two Major Approaches	50
1. Classifier Guidance	50
Deriving the Conditional Score	51
Interpretation	51

Classifier Guidance Architecture	52
Conditional Reverse Mean	52
Major Difficulty of Classifier Guidance	52
2. Classifier-Free Guidance	53
Motivation	53
Interpretation of λ	54
Training Classifier-Free Guidance	54
Conditional Mode	54
Unconditional Mode	54
CFG Sampling	55
Why CFG Became the Standard	55
Final Perspective	55
Latent Diffusion Models	56
Motivation	56
Learning a Latent Space	56
Encoder and Decoder	57
Projection into Latent Space	58
Diffusion in Latent Space	58
Inference in Latent Diffusion Models	58
Complete Latent Diffusion Pipeline	59
Why Latent Diffusion Works Better	59
Connection to Stable Diffusion	60
Final Perspective	60
Denoising Diffusion Implicit Models (DDIM)	61
Limitations of DDPM	61
1. Slow Sampling	61
2. Non-Deterministic Latent Representation	61

Implications for Representation Learning	62
Core Problem	62
Motivation for DDIMs	63
DDIMs as a Family of Inference Distributions	63
Family of Inference Distributions	63
Marginal at the Final Step	64
Non-Markovian Transitions	64
Key Property: Shared Marginals	64
Consequence for the ELBO	64
Key Insight: Many Joint Distributions, Same Marginals	65
Reverse Process in DDIM	65
Connection to Noise Prediction	66
Key Result	66
Main Theoretical Conclusion	66
Critical Insight	66
Practical Implication: Faster Sampling	67
Final Perspective	67
Inference in DDIMs	67
Reverse Mean Parameterization	67
DDIM Forward Posterior Family	68
Sampling Procedure	68
Interpretation of the Noise Parameter σ_t	68
Connection to DDPM	69
Deterministic DDIM ($\sigma_t = 0$)	69
DDIM as an Implicit Model	69

DDIM Inversion and Unique Latents	70
Implications for Image Editing	70
Final Insight	70

Introduction

Diffusion models are currently among the most powerful image generation models and achieve state-of-the-art (SOTA) performance in generative modeling.

One of the most important classes of diffusion models is the family of **Denoising Diffusion Probabilistic Models (DDPMs)**.

Mathematically, DDPMs can be understood as a special class of **Hierarchical Variational Autoencoders (HVAEs)**.

From VAE to Hierarchical VAE

Recall that in a standard Variational Autoencoder (VAE), we have a single latent representation:

$$x \rightarrow q_\phi(z|x) \rightarrow z \rightarrow p_\theta(x|z) \rightarrow x$$

where:

- x is the data
- z is the latent variable
- $q_\phi(z|x)$ is the encoder
- $p_\theta(x|z)$ is the decoder
- ϕ, θ are learnable neural network parameters

Thus, a VAE learns:

1. An encoder mapping data to latent space
2. A decoder mapping latent space back to data space

Hierarchical VAE

A Hierarchical VAE extends this idea by introducing multiple latent variables arranged across several levels:

$$x \rightarrow z_1 \rightarrow z_2 \rightarrow z_3 \rightarrow \cdots \rightarrow z_T$$

This is the **encoding** process.

The decoding process reverses this hierarchy:

$$z_T \rightarrow z_{T-1} \rightarrow \cdots \rightarrow z_1 \rightarrow x$$

The hierarchy enables the model to progressively represent information at different abstraction levels.

DDPM as a Hierarchical VAE

A DDPM can be viewed as a Hierarchical VAE with several important special properties.

1. Multiple Latent Spaces

Instead of a single latent variable, DDPMs contain a sequence of latent variables:

$$z_1, z_2, \dots, z_T$$

Typically, the notation used in diffusion literature is:

$$x_0, x_1, x_2, \dots, x_T$$

where:

- x_0 is the original data
- x_T is nearly pure Gaussian noise

2. Latent Dimension Equals Data Dimension

Unlike standard VAEs, diffusion models do not compress the data into a smaller latent space.

For every timestep:

$$\dim(z_t) = \dim(x)$$

Thus, every latent variable has exactly the same dimensionality as the input image.

For example:

- If the image is $64 \times 64 \times 3$
- then every z_t also has shape $64 \times 64 \times 3$

This is a major conceptual difference from classical latent variable models.

3. The Encoder (Forward Process) is Fixed

In a VAE, the encoder is learned:

$$q_\phi(z|x)$$

where the parameters ϕ are optimized during training.

In DDPMs, the forward process is **not learned**.

Instead, it is a predefined stochastic corruption process:

$$q(x_t|x_{t-1})$$

that gradually adds Gaussian noise to the data.

The forward diffusion process is fixed beforehand and typically defined as:

$$q(x_t|x_{t-1}) = \mathcal{N}\left(x_t; \sqrt{1 - \beta_t}x_{t-1}, \beta_t I\right)$$

where:

- β_t is a predefined noise schedule
- Gaussian noise is progressively injected at each timestep

Thus:

- the encoder is known analytically
- requires no neural network
- and has no learnable parameters

4. Only the Decoder is Learned

In a standard VAE:

- encoder is learned
- decoder is learned

In DDPMs:

- forward diffusion process is fixed
- reverse denoising process is learned

The reverse process is parameterized as:

$$p_{\theta}(x_{t-1}|x_t)$$

which learns how to remove noise step-by-step.

Generation begins from pure Gaussian noise:

$$x_T \sim \mathcal{N}(0, I)$$

and repeatedly denoises:

$$x_T \rightarrow x_{T-1} \rightarrow \cdots \rightarrow x_0$$

until a realistic sample is generated.

Summary

DDPMs can therefore be interpreted as:

- hierarchical latent variable models
- with many latent spaces
- where each latent space has the same dimension as the data
- with a fixed forward diffusion process
- and a learned reverse denoising process

This perspective connects diffusion models naturally to probabilistic graphical models and variational inference.

DDPM Formulation

In a DDPM, there is no learnable encoding process.

- **Forward Process** $\rightarrow$ Noising Process
- **Reverse Process** $\rightarrow$ Denoising Process

The forward process is fixed and non-learnable.

No neural network is involved in constructing the latent representation during the forward diffusion process.

Notation

- Data variable is denoted by x_0
- Latent variables are denoted by:

$$x_1, x_2, x_3, \dots, x_T$$

This differs from VAEs where latent variables are typically written as:

$$z_1, z_2, z_3, \dots, z_T$$

The forward diffusion process is:

$$x_0 \rightarrow x_1 \rightarrow x_2 \rightarrow \dots \rightarrow x_T$$

where:

- x_0 is the original data point
- each x_t is a noisy latent representation of x_0

For all timesteps:

$$\dim(x_t) = \dim(x_0)$$

The number of diffusion steps T is a hyperparameter.

Forward Process

The forward process is a fixed stochastic process.

Define:

$$\beta_1, \beta_2, \dots, \beta_T$$

as a predefined variance schedule.

Then define:

$$\alpha_t = 1 - \beta_t$$

The forward diffusion process is defined recursively as:

$$x_1 = \sqrt{\alpha_1}x_0 + \sqrt{1 - \alpha_1}\epsilon_1$$

where:

$$\epsilon_1 \sim \mathcal{N}(0, I)$$

Similarly,

$$x_2 = \sqrt{\alpha_2}x_1 + \sqrt{1 - \alpha_2}\epsilon_2$$

where:

$$\epsilon_2 \sim \mathcal{N}(0, I)$$

Continuing recursively:

$$x_t = \sqrt{\alpha_t}x_{t-1} + \sqrt{1 - \alpha_t}\epsilon_t$$

where:

$$\epsilon_t \sim \mathcal{N}(0, I)$$

This is the fundamental forward diffusion equation:

$$x_t = \sqrt{\alpha_t}x_{t-1} + \sqrt{1 - \alpha_t}\epsilon_t \tag{1}$$

where:

$$\alpha_t \in [0, 1]$$

are fixed scalars.

Equation 1 defines the encoding or forward diffusion process of DDPMs.

Even though this process is fixed and non-learnable, it is still probabilistic because Gaussian noise is injected at every timestep.

Forward Process as a Recurrent Equation

Observe that:

$$x_t$$

depends on:

$$x_{t-1}$$

and random noise:

$$\epsilon_t$$

Thus Equation 1 is a recurrent stochastic equation.

By recursively substituting previous equations into later ones, we obtain a closed-form expression for x_t .

Derivation of Closed Form

Starting with:

$$x_t = \sqrt{\alpha_t}x_{t-1} + \sqrt{1 - \alpha_t}\epsilon_t$$

Substitute:

$$x_{t-1} = \sqrt{\alpha_{t-1}}x_{t-2} + \sqrt{1 - \alpha_{t-1}}\epsilon_{t-1}$$

Then:

$$x_t = \sqrt{\alpha_t\alpha_{t-1}}x_{t-2} + \sqrt{\alpha_t(1 - \alpha_{t-1})}\epsilon_{t-1} + \sqrt{1 - \alpha_t}\epsilon_t$$

Repeating recursively:

$$x_t = \sqrt{\alpha_t\alpha_{t-1}\cdots\alpha_1}x_0 + \text{weighted Gaussian noise}$$

Define:

$$\bar{\alpha}_t = \prod_{s=1}^t \alpha_s$$

Then the forward process admits the closed form:

$$x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1 - \bar{\alpha}_t}\epsilon \tag{2}$$

where:

$$\epsilon \sim \mathcal{N}(0, I)$$

Equation 2 is one of the most important equations in diffusion models.

It allows direct sampling of x_t from x_0 without recursively computing all intermediate states.

Forward Process as a Markov Chain

Equation 1 defines a first-order Markov chain.

A first-order Markov chain is a sequence of random variables:

$$x_0, x_1, x_2, \dots, x_T$$

such that:

$$x_t \perp x_{t-2}, x_{t-3}, \dots, x_0 \mid x_{t-1}$$

This means:

- x_t depends only on x_{t-1}
- given x_{t-1} , all earlier states become conditionally independent

Thus the forward conditional distributions are:

$$q(x_t | x_{t-1}) = \mathcal{N}(x_t; \sqrt{\alpha_t} x_{t-1}, (1 - \alpha_t) I)$$

Equivalently:

$$x_t = \mu_t + \sqrt{\sigma_t} \epsilon_t$$

with:

$$\mu_t = \sqrt{\alpha_t} x_{t-1}$$

and

$$\sigma_t = 1 - \alpha_t$$

These conditional distributions define the forward encoding process.
As:

$$t \rightarrow T$$

the distribution converges toward pure Gaussian noise:

$$q(x_T) \approx \mathcal{N}(0, I)$$

which acts as the stationary distribution of the diffusion process.

Reverse Process

The reverse process attempts to invert the noising process.

Instead of adding noise, the reverse process removes noise progressively.

The reverse process is defined via the model distribution:

$$p_{\theta}(x_0, x_1, \dots, x_T)$$

Factorizing according to the reverse Markov chain gives:

$$p_{\theta}(x_0, x_1, \dots, x_T) = p(x_T) \prod_{t=1}^T p_{\theta}(x_{t-1}|x_t) \quad (3)$$

where:

$$p(x_T) = \mathcal{N}(0, I)$$

The reverse transition distributions are defined as Gaussian:

$$p_{\theta}(x_{t-1}|x_t) = \mathcal{N}(x_{t-1}; \mu_{\theta}(x_t, t), \Sigma_{\theta}(x_t, t))$$

Thus DDPM defines a latent variable model similar to VAEs:

$$p_{\theta}(x, z)$$

but now with many latent variables:

$$x_1, x_2, \dots, x_T$$

Unlike standard VAEs, the structure of the model distribution arises directly from the reverse Markov assumption.

Interpretation

The reverse process assumes:

- first-order Markov structure
- Gaussian transitions
- learnable mean and variance

Thus:

$$p_{\theta}(x_{t-1}|x_t) = \mathcal{N}(x_{t-1}; \mu_{\theta}(x_t, t), \Sigma_{\theta}(x_t, t))$$

where:

- $\mu_{\theta}(x_t, t)$ is learned
- $\Sigma_{\theta}(x_t, t)$ is learned or predefined

The parameters are optimized using variational inference through the Evidence Lower Bound (ELBO).

ELBO Formulation

The parameters of the reverse diffusion process in a DDPM are learned using variational inference through the Evidence Lower Bound (ELBO).

Recall that in a latent variable model:

$$p_\theta(x) = \int p_\theta(x, z) dz$$

Taking logarithm:

$$\log p_\theta(x) = \log \int p_\theta(x, z) dz$$

Introducing an arbitrary variational distribution:

$$q_\phi(z|x)$$

gives the ELBO:

$$\mathcal{J}_\theta(q_\phi) = \mathbb{E}_{q_\phi(z|x)} \left[\log \frac{p_\theta(x, z)}{q_\phi(z|x)} \right]$$

ELBO for DDPM

In a DDPM, the latent variables are:

$$x_1, x_2, \dots, x_T$$

Define:

$$X_{1:T} = (x_1, x_2, \dots, x_T)$$

and

$$X_{0:T} = (x_0, x_1, \dots, x_T)$$

Then the DDPM ELBO becomes:

$$\mathcal{J}_\theta^{\text{DDPM}}(q) = \mathbb{E}_{q(X_{1:T}|x_0)} \left[\log \frac{p_\theta(X_{0:T})}{q(X_{1:T}|x_0)} \right]$$

Explicitly:

$$\mathcal{J}_\theta^{\text{DDPM}}(q) = \mathbb{E}_{q(x_{1:T}|x_0)} \left[\log \frac{p_\theta(x_{0:T})}{q(x_{1:T}|x_0)} \right]$$

Factorization of the Reverse Process

The reverse process is modeled as a reverse-time Markov chain:

$$p_{\theta}(X_{0:T}) = p(x_T) \prod_{t=1}^T p_{\theta}(x_{t-1}|x_t)$$

where:

$$p(x_T) = \mathcal{N}(0, I)$$

The reverse transition distributions are learnable.

Factorization of the Forward Process

The forward process is a first-order Markov process:

$$q(X_{1:T}|x_0) = \prod_{t=1}^T q(x_t|x_{t-1})$$

This follows from the Markov property:

$$q(x_t|x_{t-1}, x_{t-2}, \dots, x_0) = q(x_t|x_{t-1})$$

Thus:

$$q(X_{1:T}|x_0) = q(x_1|x_0)q(x_2|x_1) \dots q(x_T|x_{T-1})$$

Substituting into the ELBO

Substituting the factorized forms:

$$\mathcal{J}_{\theta}(q) = \mathbb{E}_q \left[\log \frac{p(x_T) \prod_{t=1}^T p_{\theta}(x_{t-1}|x_t)}{\prod_{t=1}^T q(x_t|x_{t-1})} \right]$$

Define:

$$\gamma = \log \frac{p(x_T) \prod_{t=1}^T p_{\theta}(x_{t-1}|x_t)}{\prod_{t=1}^T q(x_t|x_{t-1})}$$

Separating the first term:

$$\gamma = \log \left(\frac{p(x_T)p_{\theta}(x_0|x_1)}{q(x_1|x_0)} \right) + \log \left(\frac{\prod_{t=2}^T p_{\theta}(x_{t-1}|x_t)}{\prod_{t=2}^T q(x_t|x_{t-1})} \right)$$

Why Rewrite the Forward Process?

Observe:

- p_θ progresses in reverse time
- q progresses in forward time

Mathematically, it is easier if both distributions are written in the same temporal direction.

Representing:

$$p_\theta(x_{t-1}|x_t)$$

in the forward direction is difficult because it is parameterized and unknown. However, rewriting:

$$q(x_t|x_{t-1})$$

in reverse direction is possible using Bayes' rule.

Applying Bayes Rule

Using the first-order Markov property:

$$q(x_t|x_{t-1}) = q(x_t|x_{t-1}, x_0)$$

Applying Bayes' rule:

$$q(x_t|x_{t-1}, x_0) = \frac{q(x_{t-1}|x_t, x_0)q(x_t|x_0)}{q(x_{t-1}|x_0)}$$

Thus:

$$q(x_t|x_{t-1}) = \frac{q(x_{t-1}|x_t, x_0)q(x_t|x_0)}{q(x_{t-1}|x_0)}$$

where:

$$q(x_{t-1}|x_t, x_0)$$

is the true reverse diffusion distribution.

Substituting into the ELBO

Substituting into γ :

$$\gamma = A + \log \prod_{t=2}^T \frac{p_{\theta}(x_{t-1}|x_t)}{\frac{q(x_{t-1}|x_t, x_0)q(x_t|x_0)}{q(x_{t-1}|x_0)}}$$

where:

$$A = \log \frac{p(x_T)p_{\theta}(x_0|x_1)}{q(x_1|x_0)}$$

Rearranging:

$$\gamma = A + \log \prod_{t=2}^T \frac{p_{\theta}(x_{t-1}|x_t)q(x_{t-1}|x_0)}{q(x_{t-1}|x_t, x_0)q(x_t|x_0)}$$

Splitting the logarithm:

$$\gamma = A + \log \prod_{t=2}^T \frac{p_{\theta}(x_{t-1}|x_t)}{q(x_{t-1}|x_t, x_0)} + \log \prod_{t=2}^T \frac{q(x_{t-1}|x_0)}{q(x_t|x_0)}$$

Define the last term as:

$$C = \log \prod_{t=2}^T \frac{q(x_{t-1}|x_0)}{q(x_t|x_0)}$$

Expanding:

$$C = \log \frac{q(x_1|x_0)q(x_2|x_0) \dots q(x_{T-1}|x_0)}{q(x_2|x_0)q(x_3|x_0) \dots q(x_T|x_0)}$$

Almost all terms cancel:

$$C = \log \frac{q(x_1|x_0)}{q(x_T|x_0)}$$

Thus:

$$A + C = \log \frac{p(x_T)p_{\theta}(x_0|x_1)}{q(x_1|x_0)} + \log \frac{q(x_1|x_0)}{q(x_T|x_0)}$$

which simplifies to:

$$A + C = \log p_{\theta}(x_0|x_1) + \log \frac{p(x_T)}{q(x_T|x_0)}$$

Therefore:

$$\gamma = \log p_{\theta}(x_0|x_1) + \log \frac{p(x_T)}{q(x_T|x_0)} + \log \prod_{t=2}^T \frac{p_{\theta}(x_{t-1}|x_t)}{q(x_{t-1}|x_t, x_0)}$$

Using:

$$\log \Pi = \sum \log$$

gives:

$$\gamma = \log p_\theta(x_0|x_1) + \log \frac{p(x_T)}{q(x_T|x_0)} + \sum_{t=2}^T \log \frac{p_\theta(x_{t-1}|x_t)}{q(x_{t-1}|x_t, x_0)}$$

Final ELBO

The ELBO becomes:

$$\mathcal{J}_\theta(q) = \mathbb{E}_{q(X_{1:T}|x_0)}[\gamma]$$

Since each term depends only on subsets of variables, the expectations can be simplified appropriately.

Finally:

$$\mathcal{J}_\theta(q) = \mathbb{E}_{q(x_1|x_0)} [\log p_\theta(x_0|x_1)] \tag{4}$$

$$+ \mathbb{E}_{q(x_T|x_0)} \left[\log \frac{p(x_T)}{q(x_T|x_0)} \right] \tag{5}$$

$$+ \sum_{t=2}^T \mathbb{E}_{q(x_{t-1}, x_t|x_0)} \left[\log \frac{p_\theta(x_{t-1}|x_t)}{q(x_{t-1}|x_t, x_0)} \right] \tag{6}$$

This is the variational objective optimized in DDPMs.

Rewriting the ELBO Using Conditional Expectations

Recall the ELBO obtained previously:

$$\mathcal{J}_\theta(q) = \mathbb{E}_{q(x_1|x_0)} [\log p_\theta(x_0|x_1)] \tag{7}$$

$$+ \mathbb{E}_{q(x_T|x_0)} \left[\log \frac{p(x_T)}{q(x_T|x_0)} \right] \tag{8}$$

$$+ \sum_{t=2}^T \mathbb{E}_{q(x_{t-1}, x_t|x_0)} \left[\log \frac{p_\theta(x_{t-1}|x_t)}{q(x_{t-1}|x_t, x_0)} \right] \tag{9}$$

We now rewrite the ELBO using the property of conditional expectations.

Conditional Expectation Property

Using:

$$q(x_{t-1}, x_t | x_0) = q(x_t | x_0) q(x_{t-1} | x_t, x_0)$$

we obtain:

$$\mathbb{E}_{q(x_{t-1}, x_t | x_0)}[\cdot] = \mathbb{E}_{q(x_t | x_0)} \mathbb{E}_{q(x_{t-1} | x_t, x_0)}[\cdot]$$

Applying this to the ELBO:

$$\mathcal{J}_\theta(q) = \mathbb{E}_{q(x_1 | x_0)} [\log p_\theta(x_0 | x_1)] \quad (10)$$

$$+ \mathbb{E}_{q(x_T | x_0)} \left[\log \frac{p(x_T)}{q(x_T | x_0)} \right] \quad (11)$$

$$+ \sum_{t=2}^T \mathbb{E}_{q(x_t | x_0)} \left[\mathbb{E}_{q(x_{t-1} | x_t, x_0)} \left[\log \frac{p_\theta(x_{t-1} | x_t)}{q(x_{t-1} | x_t, x_0)} \right] \right] \quad (12)$$

Writing the ELBO in Terms of KL Divergence

Recall the KL divergence definition:

$$D_{KL}(q || p) = \mathbb{E}_q \left[\log \frac{q}{p} \right]$$

Equivalently:

$$\mathbb{E}_q \left[\log \frac{p}{q} \right] = -D_{KL}(q || p)$$

Using this identity:

$$\mathcal{J}_\theta(q) = \mathbb{E}_{q(x_1 | x_0)} [\log p_\theta(x_0 | x_1)] \quad (13)$$

$$- D_{KL}(q(x_T | x_0) || p(x_T)) \quad (14)$$

$$- \sum_{t=2}^T \mathbb{E}_{q(x_t | x_0)} [D_{KL}(q(x_{t-1} | x_t, x_0) || p_\theta(x_{t-1} | x_t))] \quad (15)$$

This is the ELBO objective optimized in DDPMs.

Interpretation of the ELBO Terms

The DDPM ELBO naturally decomposes into three major components.

1. Reconstruction Term

$$\mathbb{E}_{q(x_1|x_0)} [\log p_\theta(x_0|x_1)]$$

This term resembles the reconstruction term in a VAE:

$$\mathbb{E}_{q(z|x)} [\log p_\theta(x|z)]$$

Interpretation:

- Given a slightly noisy version x_1
- the model should reconstruct the original data x_0

Thus this term encourages accurate reconstruction.

2. Prior Matching Term

$$D_{KL}(q(x_T|x_0) || p(x_T))$$

This term resembles the VAE regularization term:

$$D_{KL}(q(z|x)||p(z))$$

Interpretation:

- The terminal noisy distribution $q(x_T|x_0)$
- should match the Gaussian prior:

$$p(x_T) = \mathcal{N}(0, I)$$

Thus this term pushes the diffusion process toward pure Gaussian noise at large timesteps.

3. Denoising Consistency Term

$$\sum_{t=2}^T \mathbb{E}_{q(x_t|x_0)} [D_{KL}(q(x_{t-1}|x_t, x_0) || p_\theta(x_{t-1}|x_t))]$$

This is the new term arising because DDPM is a hierarchical latent variable model with many latent spaces.

Interpretation:

- $q(x_{t-1}|x_t, x_0)$
is the true reverse diffusion distribution
- $p_\theta(x_{t-1}|x_t)$
is the learnable reverse denoising model

This term forces the learned reverse process to match the true reverse diffusion process.

Hence this term is often interpreted as:

- denoising matching term
- consistency term
- reverse process matching term

Summary of ELBO Terms

Therefore, the DDPM ELBO contains:

1. Reconstruction term

$$\mathbb{E}_{q(x_1|x_0)}[\log p_\theta(x_0|x_1)]$$

2. Prior matching term

$$D_{KL}(q(x_T|x_0)||p(x_T))$$

3. Denoising consistency term

$$\sum_{t=2}^T \mathbb{E}_{q(x_t|x_0)} D_{KL}(q(x_{t-1}|x_t, x_0)||p_\theta(x_{t-1}|x_t))$$

Optimization Objective

The final optimization objective is:

$$\theta^* = \arg \max_{\theta} \mathcal{J}_\theta(q)$$

Equivalently, one minimizes the negative ELBO.

Computing the ELBO Terms

Recall the DDPM ELBO:

$$\mathcal{J}_\theta(q) = \mathbb{E}_{q(x_1|x_0)} [\log p_\theta(x_0|x_1)] \tag{16}$$

$$- D_{KL}(q(x_T|x_0)||p(x_T)) \tag{17}$$

$$- \sum_{t=2}^T \mathbb{E}_{q(x_t|x_0)} [D_{KL}(q(x_{t-1}|x_t, x_0)||p_\theta(x_{t-1}|x_t))] \tag{18}$$

Ignoring the Prior Matching Term

The second term:

$$D_{KL}(q(x_T|x_0)||p(x_T))$$

is independent of the learnable parameters θ .

Therefore, during optimization, it can be ignored.

Thus the important terms are:

- reconstruction term
- denoising consistency term

Computing the Reverse Posterior

We need to compute:

$$q(x_{t-1}|x_t, x_0)$$

Using Bayes' rule:

$$q(x_{t-1}|x_t, x_0) = \frac{q(x_t|x_{t-1}, x_0)q(x_{t-1}|x_0)}{q(x_t|x_0)}$$

Since the forward process is first-order Markov:

$$q(x_t|x_{t-1}, x_0) = q(x_t|x_{t-1})$$

Thus:

$$q(x_{t-1}|x_t, x_0) = \frac{q(x_t|x_{t-1})q(x_{t-1}|x_0)}{q(x_t|x_0)}$$

We reverse the forward process using Bayes' rule because the forward process is known analytically.

Known Forward Distributions

We already know:

$$q(x_t|x_{t-1}) = \mathcal{N}(x_t; \sqrt{\alpha_t}x_{t-1}, (1 - \alpha_t)I)$$

We also derived:

$$q(x_t|x_0) = \mathcal{N}(x_t; \sqrt{\bar{\alpha}_t}x_0, (1 - \bar{\alpha}_t)I)$$

where:

$$\bar{\alpha}_t = \prod_{s=1}^t \alpha_s$$

Similarly:

$$q(x_{t-1}|x_0) = \mathcal{N}(x_{t-1}; \sqrt{\bar{\alpha}_{t-1}}x_0, (1 - \bar{\alpha}_{t-1})I)$$

Deriving the Closed Form of $q(x_t|x_0)$

Starting from:

$$x_t = \sqrt{\alpha_t}x_{t-1} + \sqrt{1 - \alpha_t}\epsilon_t$$

Recursively substituting:

$$x_{t-1} = \sqrt{\alpha_{t-1}}x_{t-2} + \sqrt{1 - \alpha_{t-1}}\epsilon_{t-1}$$

gives:

$$x_t = \sqrt{\alpha_t\alpha_{t-1}}x_{t-2} + \text{Gaussian noise}$$

Continuing recursively:

$$x_t = \sqrt{\prod_{s=1}^t \alpha_s}x_0 + \text{Gaussian noise}$$

Define:

$$\bar{\alpha}_t = \prod_{s=1}^t \alpha_s$$

Then:

$$x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1 - \bar{\alpha}_t}\epsilon$$

where:

$$\epsilon \sim \mathcal{N}(0, I)$$

Therefore:

$$q(x_t|x_0) = \mathcal{N}(x_t; \sqrt{\bar{\alpha}_t}x_0, (1 - \bar{\alpha}_t)I)$$

This equation is extremely important because it allows direct sampling of x_t from x_0 without recursively traversing all intermediate timesteps.

Deriving $q(x_{t-1}|x_t, x_0)$

Using Bayes' rule:

$$q(x_{t-1}|x_t, x_0) \propto q(x_t|x_{t-1})q(x_{t-1}|x_0)$$

Substituting the Gaussian forms:

$$q(x_t|x_{t-1}) = \exp\left(-\frac{\|x_t - \sqrt{\alpha_t}x_{t-1}\|^2}{2(1 - \alpha_t)}\right)$$

and

$$q(x_{t-1}|x_0) = \exp\left(-\frac{\|x_{t-1} - \sqrt{\bar{\alpha}_{t-1}}x_0\|^2}{2(1 - \bar{\alpha}_{t-1})}\right)$$

Multiplying exponentials gives another Gaussian in x_{t-1} .
Completing the square in x_{t-1} yields:

$$q(x_{t-1}|x_t, x_0) = \mathcal{N}(x_{t-1}; \mu_q(x_t, x_0), \Sigma_q)$$

where:

$$\mu_q(x_t, x_0) = \frac{\sqrt{\alpha_t}(1 - \bar{\alpha}_{t-1})x_t + \sqrt{\bar{\alpha}_{t-1}}(1 - \alpha_t)x_0}{1 - \bar{\alpha}_t} \quad (19)$$

and

$$\Sigma_q = \frac{(1 - \alpha_t)(1 - \bar{\alpha}_{t-1})}{1 - \bar{\alpha}_t} I \quad (20)$$

Thus both:

$$\mu_q \quad \text{and} \quad \Sigma_q$$

are analytically computable and completely known.

KL Divergence in the Denoising Term

We now compute:

$$D_{KL}(q(x_{t-1}|x_t, x_0) || p_\theta(x_{t-1}|x_t))$$

Assume:

$$p_\theta(x_{t-1}|x_t) = \mathcal{N}(x_{t-1}; \mu_\theta(x_t, t), \Sigma_q)$$

where the covariance is chosen equal to the true covariance.

Thus:

$$D_{KL}(\mathcal{N}(\mu_q, \Sigma_q) || \mathcal{N}(\mu_\theta, \Sigma_q))$$

Using the KL divergence formula between Gaussians:

$$D_{KL} = \frac{1}{2} \left[\log \frac{|\Sigma_q|}{|\Sigma_q|} - d + \text{tr}(\Sigma_q^{-1} \Sigma_q) \right] \quad (21)$$

$$+ \frac{1}{2} (\mu_\theta - \mu_q)^T \Sigma_q^{-1} (\mu_\theta - \mu_q) \quad (22)$$

The first terms cancel:

$$\log 1 = 0$$

and:

$$\text{tr}(I) = d$$

Thus:

$$D_{KL} = \frac{1}{2} (\mu_\theta - \mu_q)^T \Sigma_q^{-1} (\mu_\theta - \mu_q)$$

Since:

$$\Sigma_q = \sigma_q^2 I$$

we obtain:

$$\Sigma_q^{-1} = \frac{1}{\sigma_q^2} I$$

Therefore:

$$D_{KL} = \frac{1}{2\sigma_q^2} \|\mu_\theta - \mu_q\|^2 \quad (23)$$

Thus the complicated probabilistic objective reduces to a simple regression problem.

Computing the Reconstruction Term

Now consider:

$$\log p_\theta(x_0|x_1)$$

We model:

$$p_\theta(x_0|x_1) = \mathcal{N}(x_0; \mu_\theta(x_1, 1), \sigma^2 I)$$

Therefore:

$$\log p_\theta(x_0|x_1) = \log \left[\frac{1}{(2\pi\sigma^2)^{d/2}} \exp \left(-\frac{\|x_0 - \mu_\theta(x_1, 1)\|^2}{2\sigma^2} \right) \right]$$

Ignoring constants independent of θ :

$$\log p_\theta(x_0|x_1) \propto -\frac{\|x_0 - \mu_\theta(x_1, 1)\|^2}{2\sigma^2}$$

Final DDPM Objective

Therefore the ELBO reduces to:

$$\mathcal{J}_\theta(q) = -\mathbb{E}_{q(x_1|x_0)} [\|x_0 - \mu_\theta(x_1, 1)\|^2] \quad (24)$$

$$- \sum_{t=2}^T \mathbb{E}_{q(x_t|x_0)} \left[\frac{1}{2\sigma_q^2} \|\mu_\theta(x_t, t) - \mu_q(x_t, x_0)\|^2 \right] \quad (25)$$

The optimization objective becomes:

$$\theta^* = \arg \max_{\theta} \mathcal{J}_\theta(q)$$

Equivalently:

$$\theta^* = \arg \min_{\theta} \mathcal{L}_{DDPM}$$

Neural Network Parameterization

The unknown function:

$$\mu_\theta(x_t, t)$$

is represented using a neural network.

The network takes as input:

$$(x_t, t)$$

and outputs:

$$\mu_\theta(x_t, t)$$

Since the neural network itself has no intrinsic notion of time, the timestep:

$$t$$

is converted into a vector representation called a timestep embedding.

Thus:

$$(x_t, t) \rightarrow \text{Neural Network} \rightarrow \mu_\theta(x_t, t)$$

Training therefore becomes a regression problem:

$$\text{Loss} = \|\mu_\theta - \mu_q\|^2$$

despite the underlying probabilistic derivation being highly sophisticated.

DDPM as a Regressor over the True Data

Recall the denoising consistency term obtained from the KL divergence:

$$\frac{1}{2\sigma_q^2} \|\mu_\theta(x_t, t) - \mu_q(x_t, x_0)\|^2$$

We previously derived the true reverse-process mean:

$$\mu_q(x_t, x_0) = \frac{\sqrt{\alpha_t}(1 - \bar{\alpha}_{t-1})}{1 - \bar{\alpha}_t} x_t + \frac{\sqrt{\bar{\alpha}_{t-1}}(1 - \alpha_t)}{1 - \bar{\alpha}_t} x_0 \quad (26)$$

Observe carefully that:

$$\mu_q(x_t, x_0)$$

depends linearly on:

- the noisy sample x_t
- the original clean data x_0

Parameterizing the Reverse Mean

Instead of directly learning:

$$\mu_\theta(x_t, t)$$

we can parameterize it using a prediction of the clean data.

Define:

$$\hat{x}_\theta(x_t, t)$$

as the neural network prediction of the original clean sample x_0 .

Then define:

$$\mu_\theta(x_t, t) = \frac{\sqrt{\alpha_t}(1 - \bar{\alpha}_{t-1})}{1 - \bar{\alpha}_t} x_t + \frac{\sqrt{\bar{\alpha}_{t-1}}(1 - \alpha_t)}{1 - \bar{\alpha}_t} \hat{x}_\theta(x_t, t) \quad (27)$$

Comparing Equation 26 and Equation 27:

$$x_0 \leftrightarrow \hat{x}_\theta(x_t, t)$$

Thus the neural network attempts to estimate the original clean data from the noisy sample.

Substituting into the KL Objective

Now substitute:

$$\mu_\theta(x_t, t)$$

and

$$\mu_q(x_t, x_0)$$

into the KL loss:

$$\frac{1}{2\sigma_q^2} \|\mu_\theta - \mu_q\|^2$$

The terms involving x_t cancel automatically because they are identical in both expressions.

Thus:

$$\frac{1}{2\sigma_q^2} \|\mu_\theta - \mu_q\|^2 \tag{28}$$

$$= \frac{1}{2\sigma_q^2} \left\| \frac{\sqrt{\bar{\alpha}_{t-1}}(1 - \alpha_t)}{1 - \bar{\alpha}_t} (\hat{x}_\theta(x_t, t) - x_0) \right\|^2 \tag{29}$$

Factoring out the scalar:

$$= \frac{1}{2\sigma_q^2} \left(\frac{\sqrt{\bar{\alpha}_{t-1}}(1 - \alpha_t)}{1 - \bar{\alpha}_t} \right)^2 \|\hat{x}_\theta(x_t, t) - x_0\|^2 \tag{30}$$

Thus the denoising objective becomes a simple regression loss.

Interpretation

The neural network now performs:

$$x_t \rightarrow \text{Neural Network} \rightarrow \hat{x}_\theta(x_t, t)$$

where:

$$\hat{x}_\theta(x_t, t) \approx x_0$$

In other words:

- input = noisy sample
- output = prediction of clean sample

Therefore the DDPM neural network can be interpreted as a universal denoiser operating across all noise levels.

Important Conceptual Insight

This viewpoint is extremely important.

Unlike VAEs and GANs:

- the neural network in DDPM training does not directly generate novel samples
- the network is not trained as a sampler

Instead:

- the network is trained as a regression model
- it predicts the underlying clean data from noisy observations

Thus during training:

$$x_t \rightarrow \hat{x}_\theta(x_t, t)$$

is fundamentally a supervised denoising regression task.

DDPM as a Denoising Regressor

The DDPM training process can therefore be summarized as:

1. Sample a clean data point:

$$x_0 \sim p_{\text{data}}(x)$$

2. Sample a timestep:

$$t \sim \text{Uniform}(1, T)$$

3. Generate a noisy version:

$$x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1 - \bar{\alpha}_t}\epsilon$$

4. Feed (x_t, t) into the neural network

5. Predict:

$$\hat{x}_\theta(x_t, t)$$

6. Minimize:

$$\|\hat{x}_\theta(x_t, t) - x_0\|^2$$

Thus the complicated probabilistic derivation of DDPMs ultimately reduces to a denoising regression problem.

Training DDPM

Training a DDPM consists of teaching a neural network to recover the original clean data from noisy versions of the data.

Training Procedure

Given a datapoint:

$$x_0 \sim p_{\text{data}}(x)$$

repeat until convergence:

1. Sample a timestep uniformly:

$$t \sim \text{Uniform}(1, T)$$

2. Generate a noisy sample using the forward diffusion process:

$$x_t \sim \mathcal{N}(\sqrt{\bar{\alpha}_t}x_0, (1 - \bar{\alpha}_t)I)$$

Equivalently:

$$x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1 - \bar{\alpha}_t}\epsilon_t$$

where:

$$\epsilon_t \sim \mathcal{N}(0, I)$$

3. Feed:

$$(x_t, t)$$

into the neural network.

4. Predict:

$$\hat{x}_\theta(x_t, t)$$

which estimates the original clean image x_0 .

5. Update parameters using gradient descent:

$$\theta_{k+1} \leftarrow \theta_k - \eta \nabla_\theta \mathcal{L}_\theta$$

where η is the learning rate.

Mini-batch Training

Suppose a batch contains M training samples.

The DDPM objective becomes:

$$\mathcal{L}_\theta = \sum_{m=1}^M \left\| \hat{x}_\theta(x_t^{(m)}, t^{(m)}) - x_0^{(m)} \right\|^2$$

where:

- $x_0^{(m)}$ is the m^{th} clean image
- $x_t^{(m)}$ is its noisy version
- $\hat{x}_\theta(x_t^{(m)}, t^{(m)})$ is the network prediction

Thus DDPM training reduces to standard supervised regression.

Forward and Backward Pass

Forward Pass

The forward pass computes:

$$\hat{x}_\theta(x_t, t)$$

from the noisy input:

$$(x_t, t)$$

using the neural network.

Backward Pass

The backward pass computes gradients:

$$\nabla_\theta \sum_{m=1}^M \left\| \hat{x}_\theta(x_t^{(m)}, t^{(m)}) - x_0^{(m)} \right\|^2$$

and updates the parameters using gradient descent.

Important Interpretation

The DDPM neural network is trained as a denoising regressor.

At training time:

$$x_t \rightarrow \text{Neural Network} \rightarrow \hat{x}_\theta(x_t, t)$$

where:

$$\hat{x}_\theta(x_t, t) \approx x_0$$

Thus the network learns how to remove noise at every diffusion timestep.

Why DDPM Training is Stable

One of the major reasons DDPMs achieve state-of-the-art performance in image generation is the remarkable stability of their training procedure.

Unlike other generative models:

- VAEs can suffer from posterior collapse
- GANs can suffer from mode collapse and adversarial instability

DDPMs avoid these problems because:

- training reduces to a simple supervised regression problem
- there is no adversarial optimization
- the forward process is fixed and analytically known
- the objective is well-behaved and smooth

Thus DDPM training is typically much more stable and reliable compared to GANs and VAEs.

Inference (Sampling) in DDPM

To generate a new sample:

$$x_0 \sim p_{\text{data}}(x)$$

we must traverse through the reverse diffusion process.

The reverse process progressively denoises a Gaussian noise sample:

$$x_T \rightarrow x_{T-1} \rightarrow x_{T-2} \rightarrow \cdots \rightarrow x_0$$

Thus generation in DDPMs occurs by iterative sampling from the learned reverse distributions.

Reverse Sampling Process

The reverse transitions are defined as:

$$x_{t-1} \sim p_{\theta}(x_{t-1}|x_t)$$

for:

$$t = T, T-1, \dots, 1$$

The process begins from pure Gaussian noise:

$$x_T \sim \mathcal{N}(0, I)$$

Then:

$$x_{T-1} \sim p_{\theta}(x_{T-1}|x_T)$$

followed by:

$$x_{T-2} \sim p_{\theta}(x_{T-2}|x_{T-1})$$

and so on until:

$$x_0 \sim p_{\theta}(x_0|x_1)$$

Learned Reverse Distribution

We model:

$$p_{\theta}(x_{t-1}|x_t) = \mathcal{N}(x_{t-1}; \mu_{\theta}(x_t, t), \sigma_q^2 I)$$

Thus sampling from the reverse process becomes:

$$x_{t-1} = \mu_{\theta}(x_t, t) + \sigma_q \epsilon$$

where:

$$\epsilon \sim \mathcal{N}(0, I)$$

Reverse Mean Parameterization

Recall the parameterization:

$$\mu_{\theta}(x_t, t) = \frac{\sqrt{\alpha_t}(1 - \bar{\alpha}_{t-1})}{1 - \bar{\alpha}_t} x_t + \frac{(1 - \alpha_t)\sqrt{\bar{\alpha}_{t-1}}}{1 - \bar{\alpha}_t} \hat{x}_{\theta}(x_t, t) \quad (31)$$

where:

$$\hat{x}_\theta(x_t, t)$$

is predicted by the trained neural network.

Thus the reverse process uses the neural network prediction of the clean image in order to compute the denoising transition.

Sampling Algorithm

Algorithm

1. Initialize:

$$x_T \sim \mathcal{N}(0, I)$$

2. For:

$$t = T, T-1, \dots, 1$$

perform:

- (a) Feed:

$$(x_t, t)$$

into the trained neural network.

- (b) Compute:

$$\hat{x}_{\theta^*}(x_t, t)$$

- (c) Compute:

$$\mu_{\theta^*}(x_t, t)$$

- (d) Sample:

$$x_{t-1} = \mu_{\theta^*}(x_t, t) + \sigma_q \epsilon$$

where:

$$\epsilon \sim \mathcal{N}(0, I)$$

3. Return:

$$x_0$$

as the generated sample.

Important Computational Observation

To generate a single sample in DDPMs, we must perform:

$$T$$

forward passes through the trained neural network.

This is because denoising occurs iteratively one timestep at a time.

Thus sampling requires:

$$x_T \rightarrow x_{T-1} \rightarrow x_{T-2} \rightarrow \cdots \rightarrow x_0$$

with a neural network evaluation at every step.

Why DDPM Sampling is Slow

Compared to GANs and VAEs:

- GANs generate samples in one forward pass
- VAEs generate samples in one decoder pass

However DDPMs require:

$$T$$

sequential denoising steps.

Therefore DDPMs are significantly slower during generation.

The computational cost of sampling is one of the major drawbacks of diffusion models.

Important Note on Hyperparameters

The noise schedule:

$$\alpha_1, \alpha_2, \dots, \alpha_T$$

consists of fixed scalars in:

$$[0, 1]$$

These are predefined hyperparameters chosen before training.

They determine:

- how quickly noise is added
- the stability of training
- the quality of generated samples

The corresponding cumulative quantities:

$$\bar{\alpha}_t = \prod_{s=1}^t \alpha_s$$

play a central role throughout both training and sampling.

Other interpretations of DDPMs

DDPM as a Noise Predictor

So far we understood DDPM training as a denoising regression problem.

Given:

- a clean datapoint x_0
- and a noisy version x_t

we trained a neural network to predict the original clean sample:

$$x_t \rightarrow \hat{x}_\theta(x_t, t) \approx x_0$$

We also saw another interpretation where the neural network predicts the mean of the true reverse distribution:

$$\mu_\theta(x_t, t)$$

Both formulations are mathematically equivalent.

Different Parameterizations of DDPM

The DDPM objective can be reformulated in several equivalent ways.

Common formulations include:

1. Data predictor
2. Noise predictor
3. Score predictor

Fundamentally, nothing changes probabilistically.

All these viewpoints arise from different reparameterizations of the same ELBO consistency term.

The underlying diffusion process remains identical.

Conditional Generation

One of the major strengths of diffusion models is conditional generation.

Instead of sampling from:

$$p(x)$$

we can sample from conditional distributions such as:

$$p(x|y)$$

where y may represent another modality.

Examples include:

- Text-to-image generation
- Class-conditioned image generation
- Image-to-image translation

Conceptually, this can be viewed as learning structured high-dimensional conditional distributions.

Noise Prediction Formulation

The original DDPM paper primarily uses the noise prediction formulation.

Instead of predicting:

$$x_0$$

the neural network predicts the noise added during the forward diffusion process.

Forward Process

Recall the closed-form forward diffusion equation:

$$x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1 - \bar{\alpha}_t}\epsilon_t \tag{32}$$

where:

$$\epsilon_t \sim \mathcal{N}(0, I)$$

The variable:

$$\epsilon_t$$

represents the Gaussian noise added to the clean sample.

Expressing x_0 in Terms of Noise

Rearranging Equation 32:

$$x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1 - \bar{\alpha}_t}\epsilon_t$$

Subtract:

$$\sqrt{1 - \bar{\alpha}_t}\epsilon_t$$

from both sides:

$$x_t - \sqrt{1 - \bar{\alpha}_t}\epsilon_t = \sqrt{\bar{\alpha}_t}x_0$$

Divide by:

$$\sqrt{\bar{\alpha}_t}$$

to obtain:

$$x_0 = \frac{x_t - \sqrt{1 - \bar{\alpha}_t}\epsilon_t}{\sqrt{\bar{\alpha}_t}} \quad (33)$$

Thus:

$$x_0 = f(x_t, \epsilon_t)$$

Consistency Term

Recall the consistency term:

$$\|\mu_\theta - \mu_q\|^2$$

where:

- μ_q is the true reverse-process mean
- μ_θ is the learned reverse-process mean

We previously derived:

$$\mu_q(x_t, x_0) = \frac{\sqrt{\alpha_t}(1 - \bar{\alpha}_{t-1})}{1 - \bar{\alpha}_t}x_t + \frac{(1 - \alpha_t)\sqrt{\bar{\alpha}_{t-1}}}{1 - \bar{\alpha}_t}x_0 \quad (34)$$

We now rewrite this entirely in terms of:

$$x_t \quad \text{and} \quad \epsilon_t$$

Substituting x_0 into

Substitute Equation 33 into Equation 34:

$$\mu_q(x_t, \epsilon_t) = \frac{\sqrt{\alpha_t}(1 - \bar{\alpha}_{t-1})}{1 - \bar{\alpha}_t} x_t \quad (35)$$

$$+ \frac{(1 - \alpha_t)\sqrt{\bar{\alpha}_{t-1}}}{1 - \bar{\alpha}_t} \left(\frac{x_t - \sqrt{1 - \bar{\alpha}_t}\epsilon_t}{\sqrt{\bar{\alpha}_t}} \right) \quad (36)$$

Since:

$$\bar{\alpha}_t = \alpha_t \bar{\alpha}_{t-1}$$

we have:

$$\sqrt{\bar{\alpha}_t} = \sqrt{\alpha_t} \sqrt{\bar{\alpha}_{t-1}}$$

Thus:

$$\frac{\sqrt{\bar{\alpha}_{t-1}}}{\sqrt{\bar{\alpha}_t}} = \frac{1}{\sqrt{\alpha_t}}$$

Substituting:

$$\mu_q(x_t, \epsilon_t) = \frac{\sqrt{\alpha_t}(1 - \bar{\alpha}_{t-1})}{1 - \bar{\alpha}_t} x_t \quad (37)$$

$$+ \frac{1 - \alpha_t}{(1 - \bar{\alpha}_t)\sqrt{\alpha_t}} x_t \quad (38)$$

$$- \frac{(1 - \alpha_t)}{(1 - \bar{\alpha}_t)\sqrt{\alpha_t}} \sqrt{1 - \bar{\alpha}_t} \epsilon_t \quad (39)$$

Combining the x_t coefficients:

$$\mu_q(x_t, \epsilon_t) = \frac{1}{\sqrt{\alpha_t}} x_t - \frac{1 - \alpha_t}{\sqrt{\alpha_t} \sqrt{1 - \bar{\alpha}_t}} \epsilon_t$$

Thus:

$$\mu_q(x_t, \epsilon_t) = \frac{1}{\sqrt{\alpha_t}} x_t - \frac{1 - \alpha_t}{\sqrt{\alpha_t}(1 - \bar{\alpha}_t)} \epsilon_t \quad (40)$$

Parameterizing the Reverse Mean Using Noise Prediction

Now define the neural network prediction:

$$\hat{\epsilon}_\theta(x_t, t)$$

Then parameterize:

$$\mu_\theta(x_t, t) = \frac{1}{\sqrt{\alpha_t}}x_t - \frac{1 - \alpha_t}{\sqrt{\alpha_t(1 - \bar{\alpha}_t)}}\hat{\epsilon}_\theta(x_t, t) \quad (41)$$

Consistency Term in Noise Space

Substituting Equations 40 and 41:

$$\|\mu_q - \mu_\theta\|^2$$

The x_t terms cancel.

Thus:

$$\|\mu_q - \mu_\theta\|^2 \propto \|\epsilon_t - \hat{\epsilon}_\theta(x_t, t)\|^2 \quad (42)$$

Therefore the DDPM objective becomes a regression problem over the added noise.

Final Noise Prediction Formulation

Training now becomes:

$$(x_t, t) \rightarrow \text{Neural Network} \rightarrow \hat{\epsilon}_\theta(x_t, t)$$

with objective:

$$\mathcal{L} = \|\epsilon_t - \hat{\epsilon}_\theta(x_t, t)\|^2$$

Interpretation

The neural network no longer predicts:

$$x_0$$

directly.

Instead, it predicts:

$$\epsilon_t$$

which is the exact Gaussian noise added during the forward diffusion process.

This is the formulation used in the original DDPM paper and in most modern diffusion architectures.

DDPM as a Score Predictor

So far we have seen multiple equivalent interpretations of DDPM training:

1. Regressor over true data
2. Regressor over reverse-process mean
3. Regressor over added noise

Another extremely important interpretation is:

DDPM as a score predictor

This viewpoint connects diffusion models with score-based generative modeling.

Tweedie's Formula

Suppose a random variable:

$$y \sim \mathcal{N}(\mu, \sigma^2 I)$$

with marginal distribution:

$$p(y)$$

Tweedie's formula states:

$$\mathbb{E}[\mu|y] = y + \sigma^2 \nabla_y \log p(y) \quad (43)$$

where:

$$\nabla_y \log p(y)$$

is called the **score function**.

Interpretation of the Score Function

The quantity:

$$\nabla_y \log p(y)$$

represents the direction in which the log-density increases most rapidly.

Thus the score function tells us:

- where the probability density is increasing
- how the distribution geometry behaves locally
- the direction toward higher likelihood regions

Derivation of Tweedie's Formula

Suppose:

$$y = \mu + \epsilon$$

where:

$$\epsilon \sim \mathcal{N}(0, \sigma^2 I)$$

Then:

$$p(y|\mu) = \frac{1}{(2\pi\sigma^2)^{d/2}} \exp\left(-\frac{\|y - \mu\|^2}{2\sigma^2}\right)$$

The marginal density is:

$$p(y) = \int p(y|\mu)p(\mu)d\mu$$

Take gradient with respect to y :

$$\nabla_y p(y) = \int \nabla_y p(y|\mu)p(\mu)d\mu$$

Now:

$$\nabla_y p(y|\mu) = -\frac{y - \mu}{\sigma^2} p(y|\mu)$$

Thus:

$$\nabla_y p(y) = -\frac{1}{\sigma^2} \int (y - \mu)p(y|\mu)p(\mu)d\mu$$

Rewrite:

$$\nabla_y p(y) = \frac{1}{\sigma^2} \int (\mu - y)p(y|\mu)p(\mu)d\mu$$

Using Bayes' rule:

$$p(\mu|y) = \frac{p(y|\mu)p(\mu)}{p(y)}$$

Therefore:

$$\nabla_y p(y) = \frac{p(y)}{\sigma^2} \int (\mu - y)p(\mu|y)d\mu$$

Divide both sides by $p(y)$:

$$\nabla_y \log p(y) = \frac{1}{\sigma^2} (\mathbb{E}[\mu|y] - y)$$

Rearranging:

$$\boxed{\mathbb{E}[\mu|y] = y + \sigma^2 \nabla_y \log p(y)}$$

This is Tweedie's formula.

Applying Tweedie's Formula to DDPM

Recall the DDPM forward distribution:

$$q(x_t|x_0) = \mathcal{N}(x_t; \sqrt{\bar{\alpha}_t}x_0, (1 - \bar{\alpha}_t)I) \quad (44)$$

Comparing with Tweedie's setting:

$$y = x_t$$

$$\mu = \sqrt{\bar{\alpha}_t}x_0$$

$$\sigma_t^2 = 1 - \bar{\alpha}_t$$

Applying Tweedie's formula:

$$\mathbb{E}[\sqrt{\bar{\alpha}_t}x_0|x_t] = x_t + (1 - \bar{\alpha}_t)\nabla_{x_t} \log p(x_t) \quad (45)$$

The best estimator of the conditional expectation is the true mean itself:

$$\mathbb{E}[\sqrt{\bar{\alpha}_t}x_0|x_t] = \sqrt{\bar{\alpha}_t}x_0$$

Thus:

$$\sqrt{\bar{\alpha}_t}x_0 = x_t + (1 - \bar{\alpha}_t)\nabla_{x_t} \log p(x_t)$$

Rearranging:

$$x_0 = \frac{1}{\sqrt{\bar{\alpha}_t}} (x_t + (1 - \bar{\alpha}_t)\nabla_{x_t} \log p(x_t)) \quad (46)$$

Thus:

$$x_0 = f(x_t, \text{score})$$

Expressing the Reverse Mean Using the Score Function

Recall:

$$\mu_q(x_t, x_0) = \frac{\sqrt{\alpha_t}(1 - \bar{\alpha}_{t-1})}{1 - \bar{\alpha}_t} x_t + \frac{(1 - \alpha_t)\sqrt{\bar{\alpha}_{t-1}}}{1 - \bar{\alpha}_t} x_0 \quad (47)$$

Substitute Equation 46:

$$\mu_q = \frac{\sqrt{\alpha_t}(1 - \bar{\alpha}_{t-1})}{1 - \bar{\alpha}_t} x_t \quad (48)$$

$$+ \frac{(1 - \alpha_t)\sqrt{\bar{\alpha}_{t-1}}}{1 - \bar{\alpha}_t} \frac{1}{\sqrt{\bar{\alpha}_t}} (x_t + (1 - \bar{\alpha}_t) \nabla_{x_t} \log p(x_t)) \quad (49)$$

Since:

$$\bar{\alpha}_t = \alpha_t \bar{\alpha}_{t-1}$$

we obtain:

$$\frac{\sqrt{\bar{\alpha}_{t-1}}}{\sqrt{\bar{\alpha}_t}} = \frac{1}{\sqrt{\alpha_t}}$$

Thus:

$$\mu_q = \frac{\sqrt{\alpha_t}(1 - \bar{\alpha}_{t-1})}{1 - \bar{\alpha}_t} x_t \quad (50)$$

$$+ \frac{1 - \alpha_t}{(1 - \bar{\alpha}_t)\sqrt{\alpha_t}} x_t \quad (51)$$

$$+ \frac{1 - \alpha_t}{\sqrt{\alpha_t}} \nabla_{x_t} \log p(x_t) \quad (52)$$

Combining coefficients of x_t :

$$\mu_q = \frac{1}{\sqrt{\alpha_t}} x_t + \frac{1 - \alpha_t}{\sqrt{\alpha_t}} \nabla_{x_t} \log p(x_t)$$

Therefore:

$$\boxed{\mu_q = \frac{1}{\sqrt{\alpha_t}} x_t + \frac{1 - \alpha_t}{\sqrt{\alpha_t}} \nabla_{x_t} \log p(x_t)} \quad (53)$$

Learned Score Function

Define a neural network:

$$s_\theta(x_t, t)$$

which predicts the score function.

Parameterize:

$$\mu_\theta(x_t, t) = \frac{1}{\sqrt{\alpha_t}}x_t + \frac{1 - \alpha_t}{\sqrt{\alpha_t}}s_\theta(x_t, t) \quad (54)$$

Consistency Term in Score Space

Now:

$$\|\mu_q - \mu_\theta\|^2$$

becomes:

$$\left\| \frac{1 - \alpha_t}{\sqrt{\alpha_t}} (\nabla_{x_t} \log p(x_t) - s_\theta(x_t, t)) \right\|^2$$

Thus:

$$\boxed{\|\mu_q - \mu_\theta\|^2 \propto \|\nabla_{x_t} \log p(x_t) - s_\theta(x_t, t)\|^2} \quad (55)$$

Therefore DDPM training can be interpreted as score matching.

True Score and Noise Relationship

Recall:

$$x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1 - \bar{\alpha}_t}\epsilon_t$$

Rearranging:

$$x_0 = \frac{x_t - \sqrt{1 - \bar{\alpha}_t}\epsilon_t}{\sqrt{\bar{\alpha}_t}} \quad (56)$$

We also derived:

$$x_0 = \frac{1}{\sqrt{\bar{\alpha}_t}} (x_t + (1 - \bar{\alpha}_t)\nabla_{x_t} \log p(x_t))$$

Equating both expressions:

$$x_t - \sqrt{1 - \bar{\alpha}_t}\epsilon_t = x_t + (1 - \bar{\alpha}_t)\nabla_{x_t} \log p(x_t)$$

Thus:

$$-\sqrt{1-\bar{\alpha}_t}\epsilon_t = (1-\bar{\alpha}_t)\nabla_{x_t}\log p(x_t)$$

Therefore:

$$\boxed{\nabla_{x_t}\log p(x_t) = -\frac{1}{\sqrt{1-\bar{\alpha}_t}}\epsilon_t} \quad (57)$$

Important Insight

The true score function is simply negatively scaled Gaussian noise.

Thus:

$$\text{score} \propto -\text{noise}$$

This explains why:

- predicting noise
- predicting score

are mathematically equivalent objectives.

Important Observation

Higher noise levels imply:

$$1 - \bar{\alpha}_t$$

is larger.

Therefore:

$$\|\nabla_{x_t}\log p(x_t)\|$$

becomes smaller.

Hence:

- highly noisy samples have smaller scores
- cleaner samples have sharper score structure

Four Interpretations of DDPM

A DDPM neural network can therefore be interpreted in four equivalent ways:

$$(x_t, t) \rightarrow \hat{x}_\theta(x_t, t) \quad \text{Regressor over true data} \quad (58)$$

$$(x_t, t) \rightarrow \mu_\theta(x_t, t) \quad \text{Regressor over reverse mean} \quad (59)$$

$$(x_t, t) \rightarrow \hat{\epsilon}_\theta(x_t, t) \quad \text{Regressor over noise} \quad (60)$$

$$(x_t, t) \rightarrow s_\theta(x_t, t) \quad \text{Regressor over score} \quad (61)$$

Final Remark

Among these formulations, the most widely used in practice is:

Noise prediction

used in the original DDPM paper and most modern diffusion architectures.

However, irrespective of interpretation, the neural network is fundamentally learning the same underlying structure because all objectives are scaled and shifted versions of the same ELBO consistency term.

Guided Diffusion Models

So far we studied DDPMs as models that sample from the marginal data distribution:

$$p(x_0)$$

However, modern diffusion models are primarily used for **conditional generation**.

Examples include:

- Text-to-image generation
- Class-conditioned image generation
- Image-conditioned generation
- Audio-conditioned generation

In conditional generation, we do not want to sample arbitrary data points. Instead, we want to sample from a conditional distribution:

$$p(x_0|y)$$

where:

- x_0 is the generated data

- y is some conditioning information

Examples:

- y = class label
- y = text embedding
- y = another image

Objective of Conditional Diffusion Models

Given training data:

$$(x_0, y)$$

the objective becomes:

Sample from $p(x_0|y)$

rather than:

$$p(x_0)$$

Two Major Approaches

There are two major approaches for conditional diffusion:

1. Classifier Guidance
2. Classifier-Free Guidance

1. Classifier Guidance

Recall that DDPMs can be interpreted as score predictors.

The unconditional DDPM predicts:

$$\nabla_{x_t} \log p(x_t)$$

which is the score of the marginal distribution.

However for conditional generation we require the conditional score:

$$\nabla_{x_t} \log p(x_t|y)$$

Deriving the Conditional Score

Using Bayes' rule:

$$p(x_t|y) = \frac{p(x_t)p(y|x_t)}{p(y)}$$

Taking logarithm:

$$\log p(x_t|y) = \log p(x_t) + \log p(y|x_t) - \log p(y)$$

Taking gradient with respect to x_t :

$$\nabla_{x_t} \log p(x_t|y) = \nabla_{x_t} \log p(x_t) \tag{62}$$

$$+ \nabla_{x_t} \log p(y|x_t) \tag{63}$$

$$- \nabla_{x_t} \log p(y) \tag{64}$$

Since:

$$p(y)$$

is independent of x_t :

$$\nabla_{x_t} \log p(y) = 0$$

Thus:

$$\boxed{\nabla_{x_t} \log p(x_t|y) = \nabla_{x_t} \log p(x_t) + \nabla_{x_t} \log p(y|x_t)} \tag{65}$$

Interpretation

The conditional score decomposes into:

1. Unconditional score:

$$\nabla_{x_t} \log p(x_t)$$

2. Classifier gradient:

$$\nabla_{x_t} \log p(y|x_t)$$

The second term is not a score function.

It comes from a classifier predicting the conditioning label.

Classifier Guidance Architecture

The diffusion model predicts the unconditional score:

$$(x_t, t) \rightarrow \hat{\epsilon}_\theta(x_t, t)$$

or equivalently:

$$\nabla_{x_t} \log p(x_t)$$

A separate classifier is trained:

$$x_t \rightarrow p(y|x_t)$$

The classifier predicts the label from noisy images.
During sampling, the classifier gradient:

$$\nabla_{x_t} \log p(y|x_t)$$

is added to guide generation toward the desired condition.

Conditional Reverse Mean

Recall the unconditional reverse mean:

$$\mu_\theta(x_t, t) = \frac{1}{\sqrt{\alpha_t}} x_t - \frac{1 - \alpha_t}{\sqrt{\alpha_t(1 - \bar{\alpha}_t)}} \hat{\epsilon}_\theta(x_t, t) \quad (66)$$

Replacing the unconditional score with the conditional score:

$$\nabla_{x_t} \log p(x_t|y)$$

gives the guided reverse process.

Sampling proceeds as:

$$x_{t-1} = \mu_\theta(x_t, t) + \sigma_q \epsilon$$

where:

$$\epsilon \sim \mathcal{N}(0, I)$$

Major Difficulty of Classifier Guidance

The effectiveness of classifier guidance depends entirely on the quality of the classifier.

However training such a classifier is difficult.

The classifier must correctly predict labels from images corrupted at all noise levels.

For example:

$$x_0 \rightarrow \text{human face} \tag{67}$$

$$x_1 \rightarrow \text{human face} \tag{68}$$

$$x_2 \rightarrow \text{human face} \tag{69}$$

$$\vdots \tag{70}$$

$$x_T \rightarrow \text{human face} \tag{71}$$

But highly noisy images contain almost no semantic information.
 Thus classification becomes extremely difficult for large t .
 This makes classifier guidance challenging and unstable.

2. Classifier-Free Guidance

Modern diffusion systems primarily use:

Classifier-Free Guidance (CFG)

instead of explicit classifiers.

Motivation

Recall:

$$\nabla_{x_t} \log p(x_t|y) = \nabla_{x_t} \log p(x_t) + \nabla_{x_t} \log p(y|x_t)$$

Rearranging:

$$\nabla_{x_t} \log p(y|x_t) = \nabla_{x_t} \log p(x_t|y) - \nabla_{x_t} \log p(x_t)$$

Substitute back:

$$\nabla_{x_t} \log p(x_t|y) \approx \nabla_{x_t} \log p(x_t) \tag{72}$$

$$+ \lambda (\nabla_{x_t} \log p(x_t|y) - \nabla_{x_t} \log p(x_t)) \tag{73}$$

Simplifying:

$\nabla_{x_t} \log p(x_t|y) = \lambda \nabla_{x_t} \log p(x_t|y) + (1 - \lambda) \nabla_{x_t} \log p(x_t)$

(74)

where:

$$\lambda$$

is the guidance strength hyperparameter.

Interpretation of λ

- $\lambda = 0$: pure unconditional generation
- $\lambda = 1$: fully conditional generation
- $\lambda > 1$: strong guidance toward condition

In practice:

$$\lambda > 1$$

is often used to improve prompt alignment.

Training Classifier-Free Guidance

Instead of training a separate classifier, we train a single diffusion model in two modes:

Conditional Mode

Input:

$$(x_t, t, y)$$

The model predicts:

$$\nabla_{x_t} \log p(x_t | y)$$

Unconditional Mode

Input:

$$(x_t, t, \emptyset)$$

where:

$$\emptyset$$

represents a null conditioning token.

The model predicts:

$$\nabla_{x_t} \log p(x_t)$$

CFG Sampling

During inference:

$$s_{\text{guided}} = \lambda s_{\theta}(x_t, t, y) \tag{75}$$

$$+ (1 - \lambda) s_{\theta}(x_t, t, \emptyset) \tag{76}$$

Thus the model internally combines:

- conditional score
- unconditional score

without requiring an external classifier.

Why CFG Became the Standard

Classifier-free guidance became the dominant paradigm because:

- no separate classifier is required
- training is simpler
- sampling quality is significantly better
- guidance strength can be controlled easily
- works extremely well for text-to-image generation

Modern systems such as:

- Stable Diffusion
- Imagen
- DALL-E

primarily rely on classifier-free guidance.

Final Perspective

A conditional DDPM can therefore be viewed as learning:

$$p(x_0|y)$$

through conditional score estimation.

Modern diffusion models operate by manipulating score functions in high-dimensional spaces and guiding reverse diffusion toward desired conditions.

This is the core principle behind modern generative AI systems such as text-to-image synthesis.

Latent Diffusion Models

The basic idea behind Latent Diffusion Models (LDMs) is:

Build diffusion models in a learned latent space instead of raw data space

Rather than applying diffusion directly on high-dimensional data such as images, diffusion is performed on compressed latent representations learned using another encoder-decoder model.

Motivation

Images typically live in extremely high-dimensional spaces.

For example:

- a 1024×1024 RGB image contains:

$$1024 \times 1024 \times 3$$

dimensions

- this corresponds to millions of pixel values

Operating diffusion processes directly in such high-dimensional spaces creates several challenges:

- high computational cost
- slow training
- slow sampling
- difficult optimization
- learning probability distributions becomes harder

The idea of latent diffusion is therefore:

Compress first, diffuse later

Learning a Latent Space

Suppose the original data distribution is:

$$x_0 \in \mathbb{R}^d$$

where:

d

is very large.

The first step is to learn a lower-dimensional latent representation:

$$z_0 \in \mathbb{R}^k$$

where:

$$k \ll d$$

This latent space is learned using an encoder-decoder architecture.

Examples include:

- Variational Autoencoders (VAEs)
- Autoencoders
- Vector Quantized VAEs (VQ-VAEs)

Encoder and Decoder

Let:

$$E_\phi$$

denote the encoder and:

$$D_\theta$$

denote the decoder.

The encoder maps data to latent space:

$$z_0 = E_\phi(x_0) \tag{77}$$

The decoder reconstructs data from latent space:

$$\hat{x}_0 = D_\theta(z_0) \tag{78}$$

The encoder-decoder pair is trained beforehand using reconstruction objectives such as:

$$\|x_0 - \hat{x}_0\|^2$$

or VAE objectives.

Projection into Latent Space

After training the encoder-decoder model, all datapoints are projected into latent space:

$$x_0 \rightarrow z_0 = E_\phi(x_0)$$

Thus instead of modeling:

$$p(x_0)$$

directly, we model:

$$p(z_0)$$

Diffusion in Latent Space

A DDPM is now trained on latent vectors:

$$z_0$$

instead of raw images.

The forward diffusion process becomes:

$$z_t = \sqrt{\bar{\alpha}_t} z_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon_t$$

where:

$$\epsilon_t \sim \mathcal{N}(0, I)$$

The reverse process learns:

$$p_\theta(z_{t-1}|z_t)$$

Thus diffusion occurs entirely in the compressed latent space.

Inference in Latent Diffusion Models

To generate a novel datapoint:

Step 1 : Sample in Latent Space

Start from Gaussian noise:

$$z_T \sim \mathcal{N}(0, I)$$

Run reverse diffusion:

$$z_T \rightarrow z_{T-1} \rightarrow \cdots \rightarrow z_0$$

This generates a novel latent vector:

$$z_{\text{novel}}$$

Step 2 : Decode into Data Space

The decoder reconstructs the final sample:

$$x_{\text{novel}} = D_{\theta^*}(z_{\text{novel}}) \quad (79)$$

Thus:

$$x_{\text{novel}} \sim p(x_0)$$

Complete Latent Diffusion Pipeline

Training

$$x_0 \rightarrow E_{\phi} \rightarrow z_0$$

Train diffusion model on:

$$z_0$$

Inference

$$z_T \sim \mathcal{N}(0, I)$$

Reverse diffusion:

$$z_T \rightarrow z_0$$

Decode:

$$x_{\text{novel}} = D_{\theta^*}(z_0)$$

Why Latent Diffusion Works Better

Latent diffusion provides several major advantages:

- dramatically lower dimensionality
- faster training
- faster sampling
- lower memory usage
- easier distribution learning

- better scalability to large images

Instead of learning diffusion over millions of pixels, the model learns diffusion over a compact semantic representation.

Connection to Stable Diffusion

Modern systems such as:

Stable Diffusion

are based on latent diffusion models.

Stable Diffusion works by:

1. encoding images into latent representations
2. running diffusion in latent space
3. decoding the generated latent back into image space

Thus Stable Diffusion is fundamentally:

A conditional latent diffusion model

typically combined with:

- text conditioning
- classifier-free guidance
- cross-attention mechanisms

Final Perspective

Latent diffusion models combine:

- representation learning
- probabilistic generative modeling
- diffusion processes

into a unified framework.

The key idea is:

Learn diffusion where the data manifold is simpler

rather than directly operating in extremely high-dimensional raw data space.

Denoising Diffusion Implicit Models (DDIM)

Denoising Diffusion Implicit Models (DDIMs) are closely related to DDPMs in spirit.

Both frameworks construct a sequence of latent variables:

$$x_0 \rightarrow x_1 \rightarrow x_2 \rightarrow \cdots \rightarrow x_T$$

and perform sampling by reversing this stochastic process.

However, DDIMs modify the reverse process in a way that enables:

- faster sampling
- deterministic generation (if desired)

Limitations of DDPM

Despite their strong generative performance, DDPMs suffer from two key limitations.

1. Slow Sampling

DDPM sampling requires iterative denoising:

$$x_T \rightarrow x_{T-1} \rightarrow \cdots \rightarrow x_0$$

Typically:

$$T \geq 1000$$

Thus generating a single sample requires thousands of neural network evaluations.

In contrast:

- VAEs generate in one forward pass
- GANs generate in one forward pass

Therefore DDPMs are significantly slower during inference.

2. Non-Deterministic Latent Representation

The forward diffusion process is stochastic:

$$x_t = \sqrt{\alpha_t}x_0 + \sqrt{1 - \alpha_t}\epsilon_t, \quad \epsilon_t \sim \mathcal{N}(0, I) \quad (80)$$

Given a fixed x_0 , repeated applications of the forward process produce different noisy samples:

$$x_0 \rightarrow \{x_T^{(1)}, x_T^{(2)}, \dots\}$$

Thus:

- there is no unique latent representation for a given x_0
- the mapping $x_0 \rightarrow x_T$ is not deterministic

Implications for Representation Learning

This non-uniqueness becomes problematic for tasks such as:

- image editing
- feature manipulation
- semantic interpolation

For example, suppose we have an image:

a poetic scene describing a woman drying her hair after a shower (This is what I call "poetic moment")

If we want to modify only the subject (e.g., replace the poet with me) while preserving all other details, we require:

- a stable latent representation of the image
- a deterministic encoding of x_0

However, in DDPMs:

- multiple different x_T can correspond to the same x_0
- no unique latent code exists for inversion

Thus:

$$x_0 \rightarrow x_T^{(1)}, x_T^{(2)}, \dots, x_T^{(n)}, \quad n \rightarrow \infty$$

The mapping is fundamentally stochastic.

Core Problem

The key issue in DDPMs is:

No deterministic inverse mapping from $x_0 \rightarrow x_T$

This limits their use in:

- representation learning
- controllable editing
- consistent latent manipulation

Motivation for DDIMs

Denoising Diffusion Implicit Models (DDIMs) were introduced to address both major limitations of DDPMs:

1. **Slow sampling**
2. **Non-unique stochastic inversion**

DDIMs modify the reverse diffusion process to allow:

- deterministic sampling trajectories (if desired)
- fewer sampling steps without retraining

Thus DDIMs provide a more flexible interpretation of diffusion dynamics while preserving the learned data distribution.

DDIMs as a Family of Inference Distributions

Denoising Diffusion Implicit Models (DDIMs) can be understood as latent variable models, similar to DDPMs.

In any latent variable model, we have:

- observed variable: x_0
- latent variables: $x_{1:T}$

and an inference (posterior) distribution:

$$q(x_{1:T}|x_0)$$

which describes how latent variables are generated from data.

Family of Inference Distributions

In DDIMs, instead of a single forward process, we define a **family** of inference distributions:

$$q_\sigma(x_{1:T}|x_0), \quad \sigma \in \mathbb{R}^+ \tag{81}$$

This family is parameterized by a scalar σ .

We factorize it as:

$$q_\sigma(x_{1:T}|x_0) = q_\sigma(x_T|x_0) \prod_{t=2}^T q_\sigma(x_{t-1}|x_t, x_0) \tag{82}$$

Marginal at the Final Step

The terminal distribution is defined as:

$$q_\sigma(x_T|x_0) = \mathcal{N}(\sqrt{\bar{\alpha}_T}x_0, (1 - \bar{\alpha}_T)I) \quad (83)$$

which matches the marginal distribution of DDPM.

Non-Markovian Transitions

For $t > 1$, DDIM defines:

$$q_\sigma(x_{t-1}|x_t, x_0) = \mathcal{N}\left(\sqrt{\bar{\alpha}_{t-1}}x_0 + \sqrt{1 - \bar{\alpha}_{t-1} - \sigma_t^2} \cdot \frac{x_t - \sqrt{\bar{\alpha}_t}x_0}{\sqrt{1 - \bar{\alpha}_t}}, \sigma_t^2 I\right) \quad (84)$$

This transition depends explicitly on x_0 , hence:

The forward process is non-Markovian

In particular:

$$x_t \not\perp x_0 \mid x_{t-1}$$

Unlike DDPMs, where the process is strictly first-order Markov.

Key Property: Shared Marginals

Despite different transition structures, both DDPM and DDIM satisfy the same marginal:

$$q_\sigma(x_t|x_0) = \mathcal{N}(\sqrt{\bar{\alpha}_t}x_0, (1 - \bar{\alpha}_t)I) \quad (85)$$

Thus:

$$q_\sigma(x_t|x_0) = q_{\text{DDPM}}(x_t|x_0)$$

This is the crucial observation.

Consequence for the ELBO

The ELBO objective for diffusion models depends only on:

$$q(x_t|x_0)$$

not on the full joint structure:

$$q(x_{1:T}|x_0)$$

Therefore:

- DDPM and DDIM share the same ELBO
- they differ only in the choice of inference process

Hence:

$$\boxed{\text{Same } q(x_t|x_0) \Rightarrow \text{Same ELBO (up to constants)}} \quad (86)$$

Key Insight: Many Joint Distributions, Same Marginals

A fundamental observation is:

$$\boxed{\text{Different joint distributions can share identical marginals}}$$

Thus, multiple choices of:

$$q(x_{1:T}|x_0)$$

can lead to the same:

$$q(x_t|x_0)$$

This allows flexibility in designing inference processes without changing training objectives.

Reverse Process in DDIM

The learned reverse process is defined as:

$$p_\theta(x_{t-1}|x_t) \quad (87)$$

In DDIM sampling, this becomes:

- deterministic or stochastic depending on σ
- non-Markovian in nature

For $t > 1$:

$$x_{t-1} = f_\theta(x_t, t)$$

and for the final step:

$$p_\theta(x_0|x_1) = \mathcal{N}(\mu_\theta(x_1), \sigma^2 I)$$

Connection to Noise Prediction

Recall:

$$x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1 - \bar{\alpha}_t}\epsilon_t$$

Thus:

$$x_0 = \frac{1}{\sqrt{\bar{\alpha}_t}} (x_t - \sqrt{1 - \bar{\alpha}_t}\hat{\epsilon}_\theta(x_t, t)) \quad (88)$$

This defines the DDIM denoising direction.

Key Result

The DDIM mean prediction is:

$$\mu_\theta(x_t) \approx x_0$$

Thus:

$\mu_\theta(x_t)$ is an estimator of the clean sample

Main Theoretical Conclusion

The most important result is:

DDPM and DDIM optimize the same ELBO

 (89)

Therefore:

- training procedure is identical
- objective function is identical
- learned neural network is identical

Critical Insight

A single trained DDPM model implicitly defines an entire family of DDIM models.

One training process $\Rightarrow$ infinite inference processes

Practical Implication: Faster Sampling

Because DDIM defines a non-Markovian process, it allows:

- skipping intermediate timesteps
- reducing number of function evaluations
- faster generation

without retraining the model.

Final Perspective

The key conceptual shift is:

- DDPM: fixed Markov forward process
- DDIM: flexible non-Markov inference family

However:

Training remains identical; only inference changes

This explains why DDIM enables fast sampling while preserving the same learned diffusion model.

Inference in DDIMs

DDIM sampling is derived from the conditional distribution:

$$p_{\theta}(x_{t-1}|x_t)$$

In the general DDIM framework, this is defined via a family of stochastic processes parameterized by σ .

Reverse Mean Parameterization

Recall the standard DDPM noise prediction formulation:

$$x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1 - \bar{\alpha}_t}\epsilon_t \tag{90}$$

Thus the predicted clean image is:

$$x_0 = \frac{1}{\sqrt{\bar{\alpha}_t}} \left(x_t - \sqrt{1 - \bar{\alpha}_t} \hat{\epsilon}_{\theta}(x_t, t) \right) \tag{91}$$

Define:

$$\hat{x}_0 = \frac{1}{\sqrt{\bar{\alpha}_t}} \left(x_t - \sqrt{1 - \bar{\alpha}_t} \hat{\epsilon}_{\theta}(x_t, t) \right) \tag{92}$$

DDIM Forward Posterior Family

DDIM introduces a family of inference distributions:

$$q_\sigma(x_{t-1}|x_t, x_0) \quad (93)$$

given by:

$$q_\sigma(x_{t-1}|x_t, x_0) = \mathcal{N}\left(\sqrt{\bar{\alpha}_{t-1}}x_0 \quad (94)$$

$$+ \sqrt{1 - \bar{\alpha}_{t-1} - \sigma_t^2} \cdot \frac{x_t - \sqrt{\bar{\alpha}_t}x_0}{\sqrt{1 - \bar{\alpha}_t}}, \sigma_t^2 I\right) \quad (95)$$

Replacing x_0 with the neural network estimate $\hat{x}_0$:

$$q_\sigma(x_{t-1}|x_t) = \mathcal{N}\left(\sqrt{\bar{\alpha}_{t-1}}\hat{x}_0 \quad (96)$$

$$+ \sqrt{1 - \bar{\alpha}_{t-1} - \sigma_t^2} \cdot \frac{x_t - \sqrt{\bar{\alpha}_t}\hat{x}_0}{\sqrt{1 - \bar{\alpha}_t}}, \sigma_t^2 I\right) \quad (97)$$

Sampling Procedure

To sample from $p_\theta(x_{t-1}|x_t)$:

$$z \sim \mathcal{N}(0, I) \quad (98)$$

and:

$$x_{t-1} = \sqrt{\bar{\alpha}_{t-1}}\hat{x}_0 \quad (99)$$

$$+ \sqrt{1 - \bar{\alpha}_{t-1} - \sigma_t^2} \cdot \frac{x_t - \sqrt{\bar{\alpha}_t}\hat{x}_0}{\sqrt{1 - \bar{\alpha}_t}} \quad (100)$$

$$+ \sigma_t z \quad (101)$$

This is the general DDIM sampling equation.

Interpretation of the Noise Parameter σ_t

The parameter σ_t controls the stochasticity of the reverse process.

Different choices of σ_t define different generative processes:

- large $\sigma_t \rightarrow$ stochastic sampling (DDPM-like)
- intermediate $\sigma_t \rightarrow$ partially stochastic
- $\sigma_t = 0 \rightarrow$ fully deterministic

Thus:

Each choice of σ_t defines a different diffusion process

Connection to DDPM

A special choice of σ_t recovers DDPM exactly:

$$\sigma_t^2 = \frac{1 - \bar{\alpha}_{t-1}}{1 - \bar{\alpha}_t} \cdot (1 - \alpha_t) \quad (102)$$

Under this setting:

$$\boxed{\text{DDIM reduces to DDPM}}$$

Thus DDPM is a special case within the DDIM family.

Deterministic DDIM ($\sigma_t = 0$)

When:

$$\sigma_t = 0$$

the sampling equation becomes:

$$x_{t-1} = \sqrt{\bar{\alpha}_{t-1}} \hat{x}_0 \quad (103)$$

$$+ \sqrt{1 - \bar{\alpha}_{t-1}} \cdot \frac{x_t - \sqrt{\bar{\alpha}_t} \hat{x}_0}{\sqrt{1 - \bar{\alpha}_t}} \quad (104)$$

In this case:

- sampling is deterministic
- no additional noise is injected at intermediate steps

The only randomness comes from:

$$x_T \sim \mathcal{N}(0, I)$$

DDIM as an Implicit Model

When $\sigma_t = 0$, DDIM becomes an implicit generative model:

- no stochastic transitions during sampling
- deterministic trajectory from $x_T \rightarrow x_0$

This is analogous in spirit to GAN-style generation:

$$x_T \rightarrow x_0 \quad \text{deterministically}$$

DDIM Inversion and Unique Latents

A key advantage of deterministic DDIM is:

Invertibility of the diffusion trajectory

Given an image x_0 , one can compute a unique latent x_T by running the forward DDIM process.

Thus:

- each image corresponds to a unique latent under DDIM
- latent manipulation becomes well-defined

Implications for Image Editing

This enables practical editing workflows:

1. Encode image via DDIM inversion:

$$x_0 \rightarrow x_T$$

2. Modify conditioning (e.g., text prompt)
3. Run deterministic reverse diffusion:

$$x_T \rightarrow \tilde{x}_0$$

This allows controlled semantic editing while preserving structure.

Final Insight

DDIM resolves two fundamental limitations of DDPM:

- **Slow sampling** $\rightarrow$ fewer steps possible
- **Non-unique latent representation** $\rightarrow$ deterministic inversion

Thus:

DDIM = DDPM family + flexible inference + deterministic sampling option

It can be viewed as a unifying framework over stochastic and deterministic diffusion processes.